FlowMind Rapport explicatif du projet

Architecture, contenu des fichiers et fonctionnement de la simulation

Version locale examinée le 16 octobre 2026 - branche manoubi.

FlowMind simule la circulation avec SUMO et compare plusieurs façons de piloter les feux. Le système actuel permet d'utiliser des feux à durée fixe ou un agent à règles Python, puis de mesurer l'attente, les files et les émissions simulées.

L'agent choisit les phases des feux. SUMO fait évoluer les véhicules. Le programme d'évaluation enregistre les données et calcule les résultats. Le dashboard permet de commander la simulation et d'explorer les courbes dans le navigateur.

Les quatre éléments à retenir

Élément	Rôle
scenarios/	Décrit le monde à simuler : carte, véhicules et itinéraires.
flowmind/	Contient les décisions des agents, la connexion à SUMO et le calcul des résultats.
scripts/	Contient les points de lancement : serveur web, simulation, évaluation et entraînement.
dashboard/ et runs/	Le premier affiche l'interface ; le second conserve les données des expériences.

Le parcours d'une simulation

```
Navigateur : choix de la méthode et des paramètres
    -> scripts/dashboard.py : reçoit la demande
    -> flowmind/evaluation.py : organise l'expérience
    -> flowmind/env.py <-> SUMO : observe et applique
    -> agents/rules.py : choisit une phase de feu
    -> runs/ : données brutes et métriques finales
    -> navigateur : courbes et historique
```

La carte animée reste dans une fenêtre SUMO distincte du navigateur. Le dashboard est une application locale : il n'est pas nécessaire de publier un site sur Internet.

1 Arborescence et fichiers à la racine

```
5DS5_FlowMind_SUMO/
  flowmind/           logique Python du système
    agents/           contrôleurs et spécialistes
    control/          coordination prévue pour l'équipe
  scripts/            commandes de lancement
  dashboard/          interface du navigateur
  scenarios/          réseaux routiers et trafic
  sumo_rl/            code de la bibliothèque SUMO-RL
  sandbox/            démonstrations exploratoires
  tests/              vérifications du programme
  runs/               expériences et résultats générés
  docs/               documentation explicative
  output/pdf/         rapport PDF
  config.yaml         paramètres
  requirements.txt    dépendances Python
  Lancer-dashboard.cmd
```

Fichier	Contenu et rôle
config.yaml	Scénario par défaut, durées des feux, seuils, coefficients des agents et seeds d'évaluation. YAML est un format de paramètres lisible.
requirements.txt	Liste des bibliothèques à installer : SUMO, TraCI, outils de données, apprentissage et tests.
Lancer-dashboard.cmd	Se place dans le dossier du projet et lance le serveur Python du dashboard.
README.md	Présentation de l'équipe, organisation du projet et commandes de démarrage.
EVALUATION_FR.md	Instructions d'utilisation, définition des métriques et limites des comparaisons.
COMMANDS.md / IDEAS.md / Guide.md	Aide aux commandes, architecture envisagée et indications de développement. Certaines parties décrivent des objectifs futurs.
pytest.ini / .gitignore	Configuration des tests ; exclusions Git pour les résultats et autres fichiers générés.

Les chemins indiqués dans ce rapport sont relatifs à la racine du projet : C:/Users/Lenovo/Desktop/5DS/pi/5DS5_FlowMind_SUMO.

2 Le dossier scenarios et ses données

Ce dossier fournit les données d'entrée de SUMO. Il ne contient pas l'intelligence de l'agent. La carte et le trafic peuvent être utilisés avec différents contrôleurs.

Fichier	Ce qu'il décrit
3x3grid/3x3Grid2lanes.net.xml	Le réseau routier de la grille : routes, voies, jonctions, connexions et programmes de feux.
3x3grid/routes14000.rou.xml	Les véhicules, leurs heures de départ et les routes qu'ils doivent suivre. Le nom du fichier ne suffit pas à connaître le nombre de véhicules actifs à un instant.
3x3grid/3x3grid.sumocfg	La configuration SUMO qui référence la carte et le trafic de la grille, avec une période de simulation.
cologne3.net.xml	Le réseau routier du scénario Cologne.
cologne3.rou.xml	Le trafic et les itinéraires fournis pour Cologne.
cologne3.sumocfg	La configuration pour ouvrir directement le scénario Cologne dans SUMO.
README.md et document de licence	Description du dossier et informations de licence des données fournies.

Exemple simple

Un véhicule est déclaré avec un identifiant, une heure de départ et une succession de routes. SUMO détermine ensuite sa position, sa vitesse et ses interactions avec les autres véhicules et les feux.

```
<vehicle id="0" depart="0.00">
    <route edges="2Ni 1Ei 0Ei 0Wo"/>
</vehicle>
```

Deux manières de lancer le scénario

Ouvrir un fichier .sumocfg dans SUMO charge sa configuration. En revanche, le programme Python sélectionne directement les fichiers réseau et trafic, puis applique ses propres paramètres. La durée et l'heure de début ne sont donc pas nécessairement identiques dans les deux modes.

Pour comparer les contrôleurs, utiliser les mêmes fichiers d'entrée, la même durée et les mêmes seeds. Une seed reproduit les choix aléatoires ; elle ne remplace pas le fichier de trafic et ne change pas automatiquement les heures de départ.

3 Le cœur Python dans flowmind

Fichier	Contenu et responsabilité
contracts.py	Classes de données communes : LaneState, JunctionState, Claim, Event et autres types. Elles définissent les informations reçues et les réponses attendues.
env.py	Seul module principal qui dialogue avec SUMO par TraCI. Lance SUMO, découvre les feux, lit les voies et les véhicules, construit les états et applique les phases avec les transitions.
evaluation.py	Crée une expérience, instancie les agents, exécute la boucle de décision, enregistre les mesures et les décisions, puis prépare les rapports.
kpis.py	Lit les séries CSV et les trajets SUMO, calcule les indicateurs et apparie les expériences comparables. Contient aussi les fonctions d'écriture JSON et CSV.
runtime.py	Valide les modifications demandées en direct et conserve une version des règles protégée par un verrou entre les tâches du serveur.
logger.py	Fichier prévu pour un journal partagé, encore à développer. Les décisions sont actuellement écrites par evaluation.py dans decisions.csv.
__init__.py	Marque le dossier comme module Python ; ne pilote pas la simulation.

Une itération de décision

1. env.py transforme l'état de SUMO en objets JunctionState. Chaque objet décrit un carrefour : phase actuelle, durée du vert, files et état des voies.

2. evaluation.py demande à l'agent du carrefour d'exécuter act(state, events, allowed). L'agent retourne l'indice de la phase verte souhaitée.

3. env.py applique cette demande avec les transitions nécessaires. Les paramètres peuvent empêcher un changement ou une libération vers une sortie saturée.

4. Chaque seconde simulée produit des mesures. À chaque point de décision, le programme enregistre également la phase demandée, la phase appliquée et l'explication.

L'agent ne modifie pas directement la carte ou les véhicules. Son action principale est le choix de la phase du feu.

4 Comprendre agents/rules.py

RuleBasedAgent est l'agent optimiseur ajouté au projet. Il utilise des règles déterministes, sans LLM, sans modèle entraîné et sans dataset externe. Une instance distincte est créée pour chaque carrefour.

Élément du code	Rôle
__init__	Lit les coefficients et les durées ; prépare la mémoire de l'attente des directions.
wants_control	Indique que cet agent peut gérer la circulation normale. Le pilote actuel appelle directement act ; l'orchestrateur complet reste à développer.
act	Évalue les phases autorisées, applique les contraintes et retourne une phase verte.
waiting_since / last_time	Mémorisent le début d'une attente au rouge et détectent une remise à zéro de la simulation.
last_reason / explain	Conservent et retournent une explication de la dernière décision. explain est hérité de base.py.

Le score utilisé pour départager les phases

```
score = poids_files * somme(files entrantes)
      + poids_attente * somme(attentes entrantes)
      + poids_arrivees * vehicules en mouvement
      - penalite_sortie * somme(files en sortie)
```

La phase avec le score le plus élevé est favorisée. Toutefois, le score ne suffit pas : les règles ci-dessous peuvent conserver une autre phase.

Ordre des règles importantes

Conserver la phase pendant une transition et respecter le vert minimal. Écarter les phases dont toutes les sorties connues sont saturées. Donner la priorité à une direction qui attend depuis longtemps. En fin de vert maximal, chercher une autre phase disponible. Sinon, ne changer que si le gain dépasse la marge configurée.

Exemple : une direction avec dix véhicules arrêtés peut recevoir la priorité sur une direction avec deux véhicules. Une sortie encombrée ou un vert minimal encore en cours peut toutefois empêcher ce choix.

Les émissions ne figurent pas directement dans le score. L'agent cherche à limiter l'attente et les changements inutiles, ce qui peut réduire les arrêts et redémarrages. Une baisse du CO2 doit être vérifiée avec les mesures SUMO ; elle n'est pas garantie.

5 Paramètres et autres agents

Paramètre de rules	Signification
queue_weight / wait_weight	Importance des files et du temps d'attente dans le score.
approach_weight	Importance des véhicules en approche qui ne sont pas arrêtés.
downstream_weight	Pénalité liée aux véhicules arrêtés sur les voies de sortie.
switch_margin	Gain minimal justifiant un changement de phase. Une valeur élevée favorise la stabilité.
blocked_occupancy	Seuil d'occupation d'une sortie ; 0,8 représente 80 %.
max_red	Durée d'attente suivie par l'agent avant de favoriser une direction délaissée.
min_green / max_green	Durées minimale et maximale du vert. Les valeurs initiales viennent des paramètres de signaux pour les commandes classiques ; la commande interactive valide ses propres valeurs.

Les autres fichiers du dossier agents

Fichier	État et rôle
base.py	Classe Specialist et fonctions communes pour sélectionner une phase autorisée et expliquer une décision.
baselines.py	Deux contrôleurs développés : FixedTime (durées fixes) et MaxPressure (pression entre files entrantes et sortantes).
flow.py	Agent Flow : peut charger un modèle PPO/DQN ; utilise des règles de secours si aucun modèle utilisable n'est chargé.
bus.py / emergency.py	Squelettes des spécialistes bus et véhicules d'urgence. Leur logique métier reste à implémenter.
pedestrian.py / incident.py	Squelettes des spécialistes piétons et incidents. Leur présence ne signifie pas qu'ils sont fonctionnels.

Les fichiers control/orchestrator.py, control/coordinator.py et control/shield.py décrivent respectivement le choix d'un spécialiste, les échanges entre carrefours et le dispositif de sécurité prévu. Ils sont encore des squelettes. Les transitions déjà gérées dans env.py ne constituent pas le dispositif complet prévu pour les piétons.

6 Le dashboard et les scripts de lancement

Fichier	Contenu et rôle
dashboard/index.html	Le frontend : HTML pour les formulaires, CSS pour la présentation et JavaScript pour appeler le serveur et dessiner les courbes SVG. Les pages Simulation et Historique sont dans ce même fichier.
scripts/dashboard.py	Le backend local : sert l'interface, reçoit les commandes, lance la simulation dans une tâche distincte et lit les résultats pour le navigateur.
scripts/run.py	Lance directement une simulation depuis un terminal avec le contrôleur choisi ; ce script n'est pas le serveur web.
scripts/evaluate.py	Lance des essais successifs pour plusieurs seeds et contrôleurs, puis exporte les comparaisons CSV et PNG.
scripts/train.py	Entraîne l'agent Flow par PPO ou DQN. Il n'est pas nécessaire pour l'agent à règles rules.py.

Les routes du serveur

Adresse ou commande HTTP	Fonction
GET /simulation et GET /history	Ouvre les deux vues du dashboard.
GET /api/state	Retourne l'état du travail, les résultats et les paramètres courants.
GET /api/series	Retourne les mesures d'un essai sur une période donnée, pour tracer les courbes.
POST /api/start	Valide les paramètres et démarre une simulation.
POST /api/control	Demande une modification de méthode ou de règles pendant l'exécution.
POST /api/stop et GET /files/...	Demande l'arrêt ; permet aussi, par les liens de fichiers, de consulter ou télécharger les résultats.

Le serveur tourne sur 127.0.0.1:8501. Le navigateur ne dialogue pas directement avec SUMO : il passe par le backend Python, puis evaluation.py et env.py. Aucune base de données n'est requise dans cette version.

7 Données enregistrées et métriques

Chaque expérience possède un identifiant et son propre sous-dossier dans runs/. Ce dossier contient les résultats, pas le code de décision.

Fichier généré	Contenu
metadata.json	Conditions initiales : scénario, contrôleur, seed, durée, configuration et empreintes des fichiers.
timeseries.csv	Mesures à chaque seconde : trafic, files, vitesse, émissions instantanées, méthode utilisée et révision des règles.
decisions.csv	Carrefour, phase précédente, phase demandée, phase appliquée et raison de la décision.
control_events.json	Chronologie des paramètres appliqués aux expériences interactives.
tripinfo.xml / statistics.xml	Sorties officielles SUMO : trajets et émissions cumulées par véhicule ; statistiques générales.
live.json / result.json	Dernier état pour le dashboard ; état final et métriques calculées.
reports/metrics.csv et comparison.csv	Récapitulatif des expériences et comparaisons appariées.
reports/*.png	Graphiques exportés. Le navigateur dessine ses propres courbes depuis les données, sans avoir à lire ces images.

Bien distinguer les indicateurs

L’attente en courbe correspond au temps d’arrêt courant moyen des véhicules présents. L’attente moyenne finale utilise waitingTime des trajets SUMO, y compris l’attente déjà observée des véhicules encore en trajet. Ces deux mesures ne sont pas interchangeables.

Le CO2 instantané est un débit, affiché en g/s. Le CO2 final est une quantité en kg calculée à partir des totaux officiels de SUMO, trajets inachevés inclus. Les NOx et particules sont également des émissions simulées, pas des relevés de capteurs réels.

Les fichiers CSV s’ouvrent avec Python ou Excel. Les XML sont les sorties détaillées de SUMO ; les JSON conservent des paramètres et des états structurés. runs/ et les modèles entraînés sont exclus du suivi Git selon la configuration du dépôt.

8 Utilisation et interprétation des résultats

Activer ou désactiver l’agent

Dans la page Simulation, choisir « Avec l’agent optimiseur » ou « Sans agent - feux classiques », puis Démarrer. Pendant l’exécution, changer la méthode et/ou les coefficients puis cliquer sur « Appliquer pendant la simulation ». La demande est appliquée au prochain point de décision, sans remettre le trafic à zéro.

Sans agent signifie ici utiliser FixedTime, le contrôleur à durée fixe du projet. Les transitions et la connexion Python restent actives ; cela ne désactive pas la simulation.

Lire les courbes historiques

Dans Historique, sélectionner les expériences avec et sans agent, puis le début et la fin de la période à afficher. Chaque métrique possède son propre sélecteur : avec agent, sans agent ou les deux. Le même essai interactif peut être choisi dans les deux listes pour voir ses différentes périodes.

Les données sont filtrées sur la période choisie avant de limiter le tracé à 600 points. Le CSV conserve les mesures complètes. Les anciennes expériences peuvent ne pas contenir toutes les métriques ajoutées récemment.

Comparer sans confondre les expériences

Une comparaison contrôlée demande les mêmes scénario, demande de trafic, durée, seed et paramètres de protocole. Les essais interrompus, échoués ou modifiés en direct sont exclus des comparaisons automatiques entre contrôleurs fixes.

Changer de méthode dans le même essai sert à observer son comportement. Cela ne prouve pas à lui seul que la nouvelle méthode est meilleure : elle hérite du trafic laissé par la méthode précédente. Répéter des essais séparés permet une comparaison plus équitable.

Commandes utiles

```
python scripts/dashboard.py
python scripts/run.py --controller rules --gui
python scripts/evaluate.py --controllers fixed_time rules --seconds 300 --seeds 100 101
python -m pytest -q
```

Limites à connaître

L’agent à règles n’a pas besoin d’un entraînement. Le système complet avec spécialistes bus, urgence, piétons et incidents reste à intégrer. Un refus d’accès à un fichier Windows a déjà été observé pendant l’enregistrement ; en cas d’échec, conserver le chemin et le message exacts pour diagnostiquer le verrouillage.

9 Démonstrations tests et ordre de lecture

Dossier ou fichier	Rôle
sandbox/Demo.py	Démonstration exploratoire sur un carrefour, avec SUMO-RL et apprentissage Q-learning.
sandbox/Multi_Demo.py	Démonstrations exploratoires sur plusieurs carrefours ou scénarios. Ces scripts ne sont pas le pilote du dashboard actuel.
sumo_rl/	Code de la bibliothèque SUMO-RL présent dans le dépôt. Il sert de base aux démonstrations ; le système principal flowmind utilise sa propre connexion SUMO.
tests/test_contracts.py	Vérifie les structures de données communes.
tests/test_env.py	Vérifie l'environnement et son interaction avec SUMO.
tests/test_flow.py / test_rules.py	Vérifient les décisions de Flow et les contraintes de l'agent à règles.
tests/test_evaluation.py	Vérifie les calculs, unités, trajets inachevés et conditions de comparaison.
tests/test_runtime.py	Vérifie la validation et les changements pendant une simulation réelle.
tests/test_dashboard.py	Vérifie les commandes HTTP, les séries temporelles et certains cas d'erreur.

Ordre conseillé pour comprendre le code

1. scenarios/ : comprendre les données d'entrée. 2. contracts.py : comprendre la forme des informations. 3. agents/rules.py : comprendre les décisions. 4. env.py : voir leur application dans SUMO. 5. evaluation.py et kpis.py : suivre l'enregistrement et les calculs. 6. runtime.py : suivre les modifications en direct. 7. scripts/dashboard.py et dashboard/index.html : comprendre les échanges avec le navigateur.

Vocabulaire essentiel

Agent : programme qui observe et décide. Phase : combinaison de feux donnant le vert à certains mouvements. TraCI : connexion entre Python et SUMO. Métrique ou KPI : mesure de performance. Seed : paramètre de reproduction des choix aléatoires. Frontend : interface du navigateur. Backend : serveur Python. Insight : conclusion tirée des mesures.

Le contenu décrit les fichiers de la copie locale examinée. Les documents de conception expriment aussi des intentions futures ; les fonctions présentes dans le code et les fichiers encore marqués TODO doivent être distingués.